Bitwise Operators :-

These operators allow the programmer to access and manipulate individual bits within a piece of data. These operators can operate upon ints and chars but not on floats and doubles. They work at bit level.

The different Bitwise operators are :-

Bitwise Operator	Meaning
&	Bitwise AND
|	Bitwise OR
^	Bitwise XOR (Exclusive OR)
>>	Right shift
<<	Left shift
~	One's complement

1) Bitwise AND (&)

This operator operates on 2 operands. While operating upon those 2 operands, they are compared on a bit-by-bit basis. The rules are as follows:-

First bit	Second bit	First bit & Second bit
0	0	0
0	1	0
1	0	0
1	1	1

Example :-

```
void main()
{
    int a=44, b=29, c;
    c = a & b;
    printf("%d", c);
}
```

O/P :- 12

ScannedbyCamScanner

This is because:-

128	64	32	16	8	4	2	1	
0	0	1	0	1	1	0	0	(a=44)
0	0	0	1	1	1	0	1	(b=29)
0	0	0	0	1	1	0	0	(a&b)=12

2) Bitwise OR (|)

This operator also operates on two operands. While operating upon those 2 operands, they are compared on a bit-by-bit basis. The rules are as follows:-

First bit	Second bit	First bit | Second bit
0	0	0
0	1	1
1	0	1
1	1	1

Example:-

```
void main()
{
    int a=44, b=29, c;
    c= a | b;
    printf("%d", c);
}
```

O/P : 61

128	64	32	16	8	4	2	1	
0	0	1	0	1	1	0	0	(a=44)
0	0	0	1	1	1	0	1	(b=29)
0	0	1	1	1	1	0	1	(a|b)=61

ScannedbyCamScanner

3) Bitwise XOR (^)

It also operates on 2 operands. The rules are as follows:-

First bit	Second bit	First bit ^ Second bit
0	0	0
0	1	1
1	0	1
1	1	0

Example:-
```
void main()
{
    int a=44, b=29, c;
    c = a^b;
    printf("%d", c);
}
```
O/P:- 49

128	64	32	16	8	4	2	1	
0	0	1	0	1	1	0	0	(a=44)
0	0	0	1	1	1	0	1	(b=29)
0	0	1	1	0	0	0	1	(a^b)=49

4) Right Shift (>>) Operator:-

It also operates on 2 operands. It shifts each bit in its left operand to the right. The number of places the bits are shifted depends on the ~~number of~~ right operand.

Example:-
```
void main()
{
    int a=44, c;
    c = a>>2;   // Here, a is the left operand & 2 is the right operand
    printf("%d", c);
}
```
O/P: 11

ScannedbyCamScanner

128	64	32	16	8	4	2	1	
0	0	1	0	1	1	0	0	(a=44)
0	0	0	0	1	0	1	1	(a>>2) = 11

5) Left Shift (<<) Operator :-

It also operates on 2 operands. It shifts each bit in its left operand to the left. The number of places the bits are shifted depends on the right operand.

Example:-

```
void main()
{
    int a=44, c;
    c = a << 2;   //Here, a is the left operand and
                  2 " " right " .
    printf("%d", c);
}
```

O/P: 176

128	64	32	16	8	4	2	1	
0	0	1	0	1	1	0	0	(a=44)
1	0	1	1	0	0	0	0	(a<<2) = 176

6) One's complement operator (~):

On taking 1's complement of a number, all 1's present in the number are changed to 0's and all 0's are changed to 1's.

Example:-

```
void main()
{
    int a=19;

    c = ~a;
    printf("%d", c);
}
```

O/P: −20

ScannedbyCamScanner

128	64	32	16	8	4	2	1	
0	0	0	1	0	0	1	1	(a=19)
1	1	1	0	1	1	0	0	(1's complement)

sign bit = 1 means negative

Actual value is

0 0 0 1 0 1 0 0 (c = ~a) = −20.

ScannedbyCamScanner